

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..6

SESSIONS

 Copy as Markdown

## Sessions

Every endpoint method receives a `Session` object. It is the entry point for the database, cache, file storage, and messaging system. Serverpod creates and closes sessions automatically for endpoint calls. For background work, you create one yourself, most easily with `withSession`.

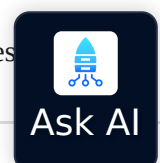
### NOTE

A Serverpod Session should not be confused with the concept of "web sessions" or "user sessions" which persist over multiple API calls. See the [Authentication documentation](#) for managing persistent authentication.

## Session types

Serverpod creates a session for every unit of work it runs, and the type reflects what triggered it. You will see these names in your [logs](#) and in Insights when tracing a call:

| Type                       | Created for                                          | Lifetime        | Typical uses                |
|----------------------------|------------------------------------------------------|-----------------|-----------------------------|
| <b>MethodCallSession</b>   | <code>Future</code> <a href="#">endpoint methods</a> | Single request  | API calls, CRUD operations  |
| <b>WebCallSession</b>      | <a href="#">Web server</a> routes                    | Single request  | Web pages, form submissions |
| <b>MethodStreamSession</b> | <a href="#">Streaming methods</a>                    | Stream duration | Real-time updates           |



| Type                     | Created for                     | Lifetime       | Typical uses                |
|--------------------------|---------------------------------|----------------|-----------------------------|
| <b>FutureCallSession</b> | <a href="#">Scheduled tasks</a> | Task execution | Email sending, batch jobs   |
| <b>InternalSession</b>   | <a href="#">Manual creation</a> | Until closed   | Background work, migrations |

You rarely choose a type yourself: your endpoint methods receive the right one, and manual creation always produces an `InternalSession`.

```
class ExampleEndpoint extends Endpoint {
  Future<String> hello(Session session, String name) async {
    // A MethodCallSession was created for this call, and it closes
    // automatically when the method returns.
    return 'Hello $name';
  }
}
```

## What a session provides

| Member                                 | What it is                                                                     |
|----------------------------------------|--------------------------------------------------------------------------------|
| <code>db</code>                        | Database access. See the <a href="#">CRUD docs</a> .                           |
| <code>caches</code>                    | Local and distributed caching. See <a href="#">caching</a> .                   |
| <code>storage</code>                   | File storage. See <a href="#">file uploads</a> .                               |
| <code>messages</code>                  | Server events for real-time communication. See <a href="#">server events</a> . |
| <code>passwords</code>                 | Secrets from config and environment. See <a href="#">configuration</a> .       |
| <code>authenticated</code>             | The current user's authentication info. See <a href="#">authentication</a> .   |
| <code>log(...)</code>                  | Write a log entry tied to this session. See <a href="#">logging</a> .          |
| <code>addWillCloseListener(...)</code> | Register a cleanup callback that runs before the session closes.               |
| <code>serverpod</code>                 | The running Serverpod instance, for example for <a href="#">future calls</a> . |

## Session lifecycle

Serverpod creates the session when the request or task starts and closes it when the work completes. Closing finalizes the session's log and releases its resources. The exception is `InternalSession`: you [create it yourself](#), so you close it yourself.

### Run cleanup when a session closes

Register callbacks that execute just before a session closes with `addWillCloseListener`. This is useful for releasing resources or performing final operations:

```
Future<void> processData(Session session) async {
  var tempFile = File('/tmp/processing_data.tmp');

  session.addWillCloseListener((session) async {
    if (await tempFile.exists()) {
      await tempFile.delete();
      session.log('Cleaned up temporary file');
    }
  });

  await tempFile.writeAsString('processing...');
  // The session closes automatically after the method returns,
  // and the cleanup callback runs.
}
```

Cleanup callbacks run in the order they were registered and fire for all session types, including manual sessions when you call `session.close()`.

## Create a session for background work

Inside an endpoint method, use the session you were given. It is already managed for you. When code runs outside a call, such as startup work in `run()` or a background job, create a session with `withSession`, which builds an `InternalSession`, runs your callback, and closes the session afterwards:

```
await pod.withSession((session) async {
  await cleanupOldRecords(session);
  await updateStatistics(session);
});
```

If the callback throws, the session is closed with the error and stack trace attached, so they reach the logs, and the error is rethrown.

If you need to manage the lifetime yourself, create the session manually and close it when done. An unclosed session is never finalized and its resources are not released, so pair `createSession` with a `finally`:

```
var session = await Serverpod.instance.createSession();
try {
  var users = await User.db.find(session);
  // Process users.
} finally {
  await session.close();
}
```

## Run work after the call returns

The session that an endpoint method receives closes when the method returns, so do not capture it in timers or delayed callbacks:

```
Future<void> processUser(Session session, int userId) async {
  Timer(Duration(seconds: 5), () async {
    // Do not do this: this session closed when processUser returned,
    // and this callback runs outside any managed lifetime.
    await updateLastSeen(session, userId);
  });
}
```

Schedule the work as a `future call` instead. Future calls survive server restarts and receive their own session when they run:

```
Future<void> processUser(Session session, int userId) async {
  await session.serverpod.futureCalls
    .callWithDelay(const Duration(seconds: 5))
    .userMaintenance
    .updateLastSeen(userId);
}
```

For one-off work that can run immediately and does not need to survive a restart, `withSession` inside the callback is the lighter alternative.

## Logging

Use `session.log` to write log entries tied to the session. Entries are persisted according to your [session log configuration](#), and closing the session finalizes its log. Sessions for [streaming methods](#) are the exception: they write entries continuously while the stream is open, so a long-lived stream appears in your logs before it ends. This is on by default in the runtime log settings. See [Logging](#) for storage, retention, and console output.

```
session.log('Processing started');
session.log('Something looks off', level: LogLevel.warning);
```

## Test with sessions

The test tools provide a `sessionBuilder` for calling endpoints in tests and simulating session state such as authentication. See [Get started with testing](#) and [Writing tests](#) for the patterns.

## Related

- [Working with endpoints](#): the methods that receive sessions.
- [Scheduling](#): future calls, the managed way to run delayed work.
- [Logging](#): where session log entries go.
- [Authentication](#): what `session.authenticated` holds.